

```
cmake --build build/  
./build/Debug/task06.exe
```

Aufgabe06

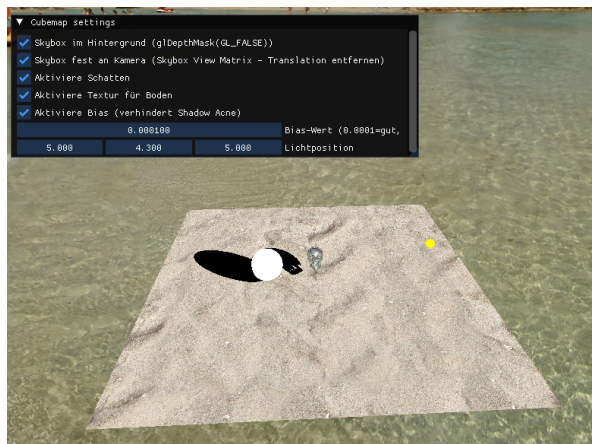
- <https://docs.gl/>
- <https://learnopengl.com/Getting-started/Textures>
- <https://developer.nvidia.com/gpugems/gpugems3/part-ii-light-and-shadows/chapter-8-summed-area-variance-shadow-maps>

6.1) Rendern in eine Textur

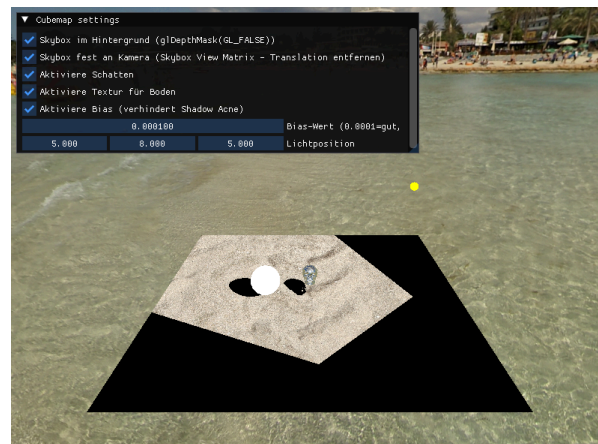
Wie erstellt man einen weiteren Framebuffer mit depth-attachment, um die Tiefeninformationen der Szene aus der Sicht der Lichtquelle zu speichern? (Shadowmap)

Implementierung

- Zuerst wird neben dem Default Framebuffer ein neuer Framebuffer erstellt, der als Shadowmap-Framebuffer dient.
- An diesen Framebuffer wird eine Textur als Depth Attachment gebunden, um die Tiefeninformationen zu speichern.
 - Dafür wird eine GL_TEXTURE_2D Textur mit dem Format GL_DEPTH_COMPONENT erstellt und an den Framebuffer gebunden
 - Besonders wichtig ist die Einstellung der Texture-Parameter, dass bei einem S/T-Wrap GL_CLAMP_TO_BORDER verwendet wird, damit außerhalb der Shadowmap ein definierter Wert (borderColor) zurückgegeben wird, der für den Tiefenvergleich im Hauptpass verwendet wird. Bei der borderColor sollte der Rot-Kanal auf 1.0 gesetzt werden, damit außerhalb der Shadowmap/ Licht-Frustum immer true ist und alles beleuchtet wird. Wenn der Rot-Kanal auf 0.0 gesetzt wird, würde außerhalb der Shadowmap alles im Schatten liegen (siehe Figure 1).



(a) Mit Border Color 1; In den Rot-Kanal wird 1.0 geschrieben, sodass beim Tiefenvergleich außerhalb der Shadowmap/ Licht-Frustum immer true ist und alles beleuchtet wird.



(b) Mit Border Color 0; Schatten außerhalb der Shadowmap/ Licht-Frustum

Figure 1: Vergleich der beiden Ergebnisse mit und ohne Rot-Kanal=1.0 beim Rendern der Shadowmap

Erstellen der Shadowmap (erster Durchlauf)

- Für den ersten Durchlauf muss das Framebuffer-Objekt (Shadowmap-Framebuffer) gebunden und die Viewportgröße auf die Größe der Shadowmap gesetzt werden, damit die Tiefenwerte korrekt in die Shadowmap geschrieben werden.
 - Für das korrekte Rendern der Szene aus der Sicht der Lichtquelle und dem Schreiben der Tiefenwerte in die Shadowmap braucht es einen eigenen Shader (shadowMapShader).
 - **Vertex-Shader:** Die Objektkoordinaten werden mit der LightCamera (LightViewMatrix und LightProjectionMatrix) transformiert, um die Szene aus der Sicht der Lichtquelle zu rendern und in `gl_Position` zu schreiben. OpenGL berechnet automatisch `gl_Position.z / gl_Position.w`, um die Tiefenwerte zu erhalten, die in der Shadowmap gespeichert werden.
 - **Fragment-Shader:** Es wird kein Fragment-Shader benötigt
 - Im ersten Durchlauf werden alle Hauptmodelle der Szene, die einen Schatten werfen sollen, mit dem shadowMapShader gerendert (Skull-Modell und Kugel).
-

6.2) Repräsentation der Lichtquelle

Nutzen Sie eine weitere Kamera, um die Position und Orientierung der Lichtquelle zu bestimmen. Rendern Sie eine Kugel und färben diese in einer gut sichtbaren Farbe ein, um die Lichtquelle in der Szene zu visualisieren.

Implementierung

- Für den Shadow Pass (erster Durchlauf) wird eine LightCamera verwendet.
 - Die Licht-Kamera wird mit der Position der Lichtquelle initialisiert und auf den Ursprung der Szene ausgerichtet.
 - An die gleiche Position wird eine gelbe Kugel (Sonne) gerendert, um die Lichtquelle in der Szene sichtbar zu machen.
 - Für das Rendern der Lichtkugel wird ein eigener Shader (lightSourceShader) verwendet, der die Kugel einfärbt, ohne diese in die Schattenberechnung einzubeziehen (sonst wäre die Oberfläche in Schatten).
-

6.4) Samplen der Shadowmap

- Im zweiten Durchlauf (Hauptpass) wird die Shadowmap als Textur gebunden und im Fragment-Shader gesampelt, um zu bestimmen, ob ein Fragment im Schatten liegt oder nicht.
 - **Vertex-Shader:** Neben den Clip-space-Koordinaten (`gl_Position`) müssen auch die Koordinaten in Licht-Sicht (`lightProj * lightView * modelMat * in_Position`) berechnet werden und an den Fragment-Shader weitergegeben.
 - **Fragment-Shader:** Die Licht-Sicht-Koordinaten werden zuerst durch `in_LightSpacePos.w` geteilt, um diese in Normalized Device Coordinates (NDC) zu transformieren. Mit dem Bias `*0.5 + 0.5` werden die NDC-Koordinaten in den Texturkoordinatenbereich `[0, 1]` transformiert.
 - Liegt die Fragmentposition außerhalb der Lichtkegel (z.B. `projCoords.z > 1.0`), wird kein Schatten berechnet, da diese Positionen nicht in der Shadowmap erfasst werden.
 - Wenn die Koordinaten im Lichtkegel liegen, werden die berechneten Texturkoordinaten verwendet, um die Shadowmap zu sampeln und den Tiefenwert der Shadowmap an dieser Stelle zu erhalten: `sampleShadowMap = texture(u_ShadowMap, projCoords.xy)`, dann `closestDepth = sampleShadowMap.r`. Standardmäßig werden die Tiefenwerte im Rot-Kanal der DepthTexture der Shadowmap gespeichert. Dieser Tiefenwert wird mit dem aktuellen Tiefenwert des Fragments `currentDepth = projCoords.z` verglichen, um zu bestimmen, ob das Fragment im Schatten liegt oder nicht: `shadow = (currentDepth - u_BiasAmount) > closestDepth ? 1.0 : 0.0;`

- Am Ende wird die Texturfarbe mit dem Schattenfaktor multipliziert, um die endgültige Farbe des Fragments zu berechnen: $\text{outColor} = \text{texColor} * (1 - \text{shadow})$.
-

6.5) Ein-/Ausschalten der Shadowmap

- Fügen Sie via **ImGui** eine **ImGui::Checkbox** hinzu, die es Ihnen erlaubt, den Schatten zu aktivieren bzw. zu deaktivieren.
- Fügen Sie eine weitere **ImGui::Checkbox** hinzu, welche die Texturierung des Bodens an/ausschaltet. Bei ausgeschalteter Textur soll der Boden in weiss gerendert werden, sodass der Schatten gut sichtbar ist.
- Fügen Sie ein **ImGui::DragFloat3**-Element hinzu, das es Ihnen ermöglicht, die Lichtquelle entlang der Weltkoordinaten Achsen zu verschieben.

Implementierung

- Im Hauptpass (zweiter Durchlauf) wird eine Uniform `u_UseShadow` hinzugefügt, die über die ImGui Checkbox gesteuert wird, um den Schatten zu aktivieren oder zu deaktivieren. Im Fragment-Shader wird dann der Schattenfaktor nur berechnet und angewendet, wenn `u_UseShadow true` ist.
 - Mit Uniform `u_UseFloorTexture` kann die Texturierung des Bodens aktiviert oder deaktiviert werden. Wenn die Texturierung deaktiviert ist, wird der Boden in weiß gerendert, um den Schatten besser sichtbar zu machen.
 - Die Position der Lichtquelle wird über eine ImGui DragFloat3 gesteuert, die die `lightPos`-Variable aktualisiert und damit die `lightViewMatrix` beeinflusst und damit die Position der Lichtquelle sowie der Shadowmap verändert.
-

6.6) Renderingartefakte beim Schattenwurf

Sie werden auf einige Renderingartefakte beim Schattenwurf stoßen. Dokumentieren Sie, wie man diese nennt, wie sie entstehen und wie diesen entgegengewirkt werden kann.

Renderingartefakt 1.: Shadow Acne

- Tiefenwerte in der Schattenkarte haben eine begrenzte Präzision, was zu stufenweiser Schattenbildung führen kann (sogenanntes “Shadow Acne”).
- Um dies zu verhindern, kann man die Präzision der Shadowmap `SHADOW_MAP_SIZE` erhöhen oder einen Tiefen-Bias hinzufügen, um die Tiefenwerte der Schattenkarte leicht zu erhöhen, damit sie weniger mit den Tiefenwerten der Szene kollidieren.
- Shadow Acne: Objekt ist gleichzeitig Schattenwerfend (im Shadow Pass) und Schattenempfänger (im Hauptpass). Durch die begrenzte Präzision der Shadowmap können schräge Geometrien zur Lichtquelle hin zu stufenweiser Schattenbildung führen, da die Tiefenwerte der Shadowmap nicht genau mit den Tiefenwerten der Szene übereinstimmen. Dies führt dazu, dass stufenweise Fragmente als im Schatten liegen interpretiert werden, obwohl diese eigentlich beleuchtet sein sollten, was zu einem Muster auf der Oberfläche führt (siehe Figure 2).

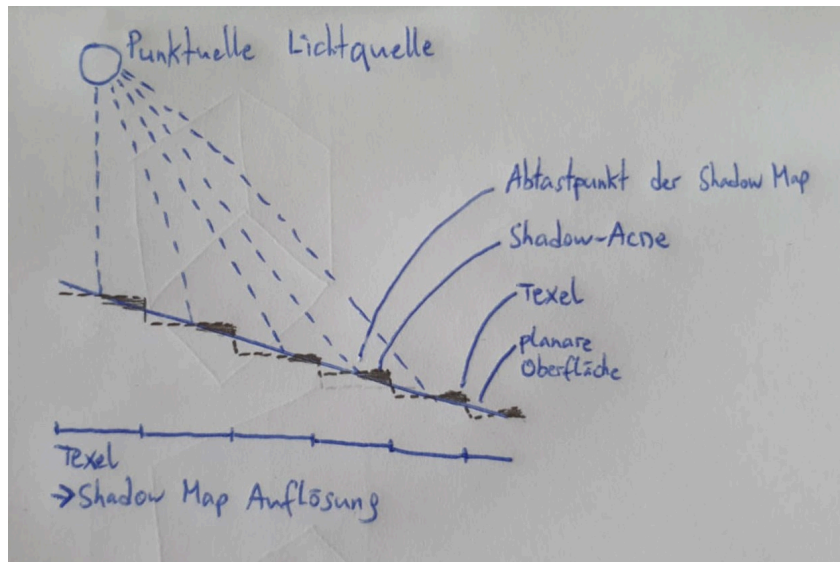


Figure 2: Zeichnung der Schattenbildung zwischen kontinuierlicher Oberfläche und diskreter Abtastung der Shadowmap.

Lösung: Tiefen-Bias

- Tiefen-Bias hinzufügen, um die Tiefenwerte der Shadowmap leicht zu erhöhen, damit sie nicht mit den Tiefenwerten der Szene kollidieren.

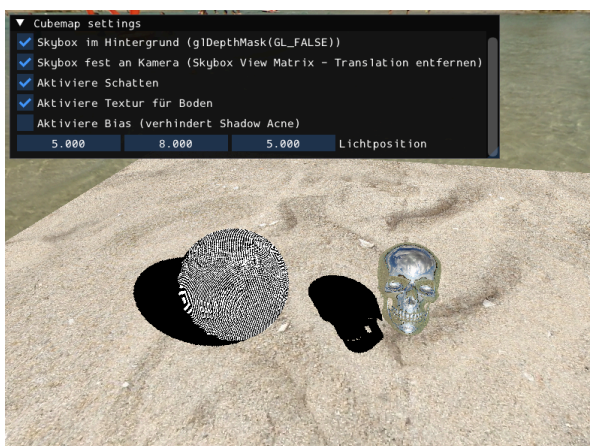
Renderingartefakt 2.: Peter Panning Effekt

- Peter Panning Effekt: Schatten driftet von der Oberfläche weg
 - Ursache: Zu großer Bias-Wert, wodurch sich der Schatten von der Oberfläche weg bewegt.
 - Lösung: Den Bias-Wert richtig einstellen, um ein Gleichgewicht zwischen Shadow Acne und Peter Panning zu finden.

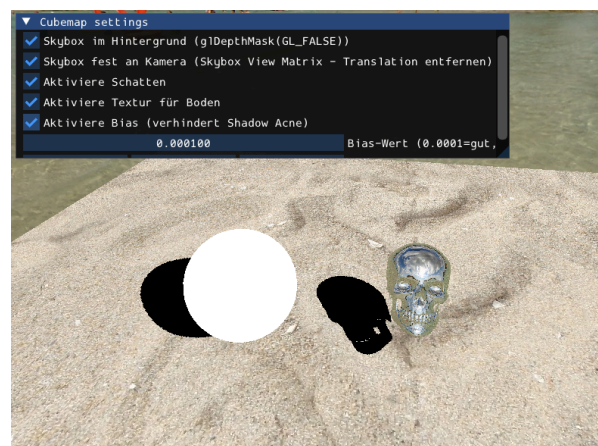
Renderingartefakt 3.: Schatten hinter der Lichtquelle

- Hinter der LightCamera liegt nur Schatten
 - $\text{if (projCoords.z} \leq 1.0) \{ \text{Schatten berechnen} \} \text{ else } \{ \text{kein Schatten} \}$
 - Damit wird außerhalb der Lichtkegel kein Schatten berechnet.

Ergebnis



(a) Shadowmapping ohne Tiefen-Bias



(b) Shadowmapping mit Tiefen-Bias

Figure 3: Vergleich zwischen mit und ohne Tiefen-Bias beim Shadowmapping